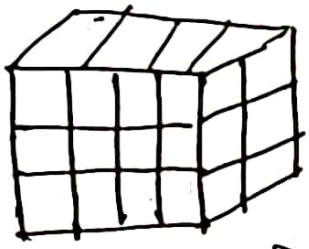
05-06-23

MULTI-DIMENSIONAL ARRAY

Linear array =



4x4x2 → Dimension

row ↙ column ↙ height ↓.

	0	1	2	3
0	0,0		0,2	
1				1,3

2x4 → Dimension

row ↙ column ↓

	0	1	2	3
0	4	5	8	9
1	1	3	6	7

m =

m = np.array([[4,5,8,9], [1,3,6,7]])

- 2ti array
- 4ti element.

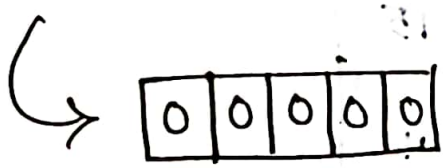
m =

	0	1	2	3	4
0	4	5	8	9	15
1	1	3	6	7	16
2	0	1	2	3	1

```
m = np.array([[4,5,8,9,15], [1,3,6,7,16],
              [0,1,2,3,1]])
```

(*) (**)

```
a = np.array([None]*5)
a = np.zeros((5))
```



(*) `m = np.zeros((3,5))`

Row

column

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0

Salicid™

(*) m.shape → print (m.shape)

[result will be a tuple containing row and column numbers].

Output:

Ex: (3,5)

* t = m.shape

row, column = t.

(*) print (m.shape)

(3,5)

	0	1	2	3	4
0	0,0 → 4	0,1 → 5	0,2 → 8	0,3 → 9	0,4 → 15
1	1,0 → 1	1,1 → 3	1,2 → 6	1,3 → 7	1,4 → 16
2	2,0 → 0	2,1 → 1	2,2 → 2	2,3 → 3	2,4 → 1

Indexing multi-dimensional array and print it:

```
for i in range (row):
    for j in range (col):
        print (m[i] [j], end = ' ')
    print ()
```

(#)

4	1	0
5	3	1
8	6	2
9	7	3
15	16	1

→

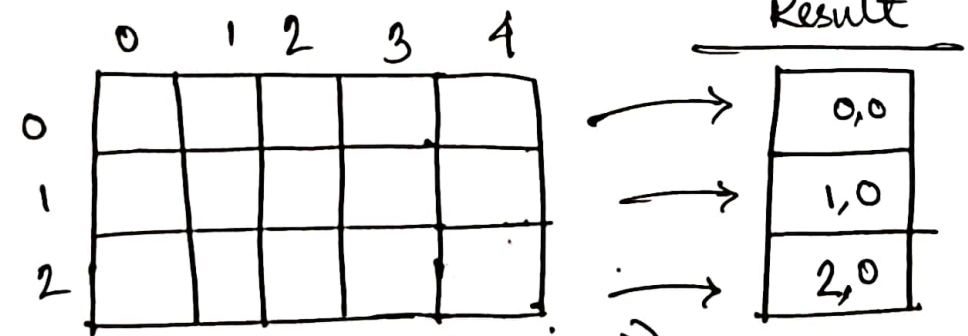
(#)

```
for i in range(col):
    for j in range(row):
        print(m[j][i], end=' ')
    print()
```

for sum

```
for i in range(col):
    for j in range(row):
        sum += m[i][j]
print()
```

Over




```
np.zeros = ((row, 1))
for i in range(row):
    for j in range(colum):
```

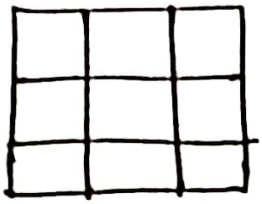
P.T.O.

Salicid™

(*)

```
row, column = m.shape.
result = np.zeroes ((row, 1))
for i in range (row):
    sum = 0
    for j in column (col):
        sum += m[i][j]
    result [i][0] = sum.
```

Ques. Practice

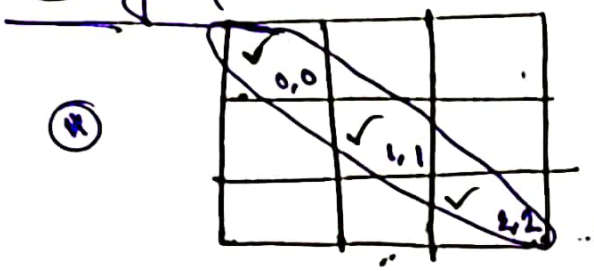
m1 = 

m2 = 

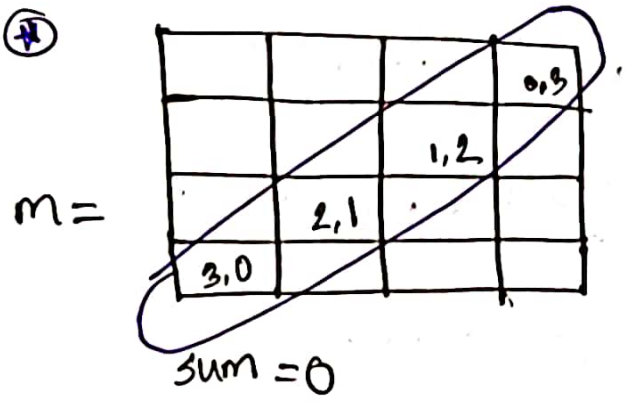
```
row, column = m.shape.
result = np.array((row),(col))
for i in range (row):
    sum = 0
    for j in range (column):
        sum += m1[i][j] + m2[i][j]
    result[i][j] = sum.
```

Salicid™

Diagonal:




```
sum = 0
for i in range (row):
    sum += m[i][i]
```



sum = 0

```
for i in range (row):          (col-1, -1, -1):
    for j in range (column-1):
        sum+= (m[i]+1) + (m[j] - 1)
```

OR

```
for i in ____ (row):
    for j ____ (col):
        if i+j == row-1:
            sum+= m[i][j]
```

OR

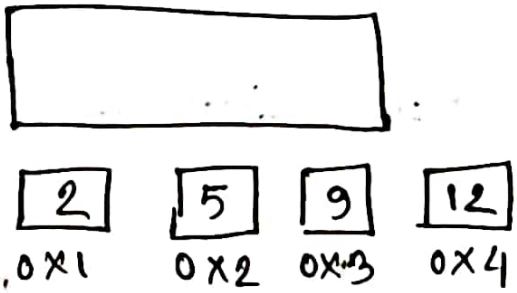
```
for i in ——— (row):
    sum = 0
    ~~sum+=~~
    j = row - 1 - j
    sum += m[i][j]
```

Base Address:

address of the 1st element.

memory representation:

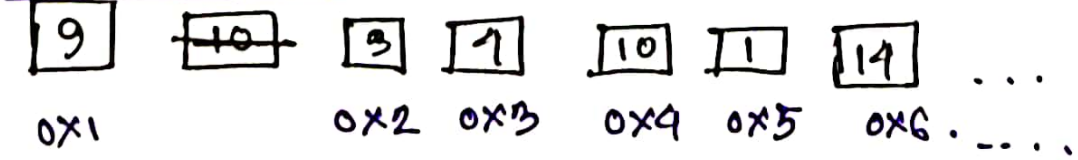
9	10	11
3	1	2
4	14	9



Row major



Column major:



Salicid™

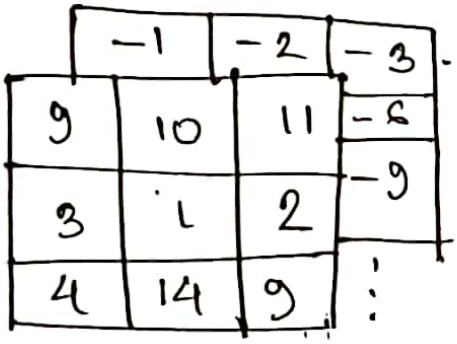
row major

elements will be row-wise distributed.

column major

will be column-wise.

3-dimensional



dimension

[4x4x2]

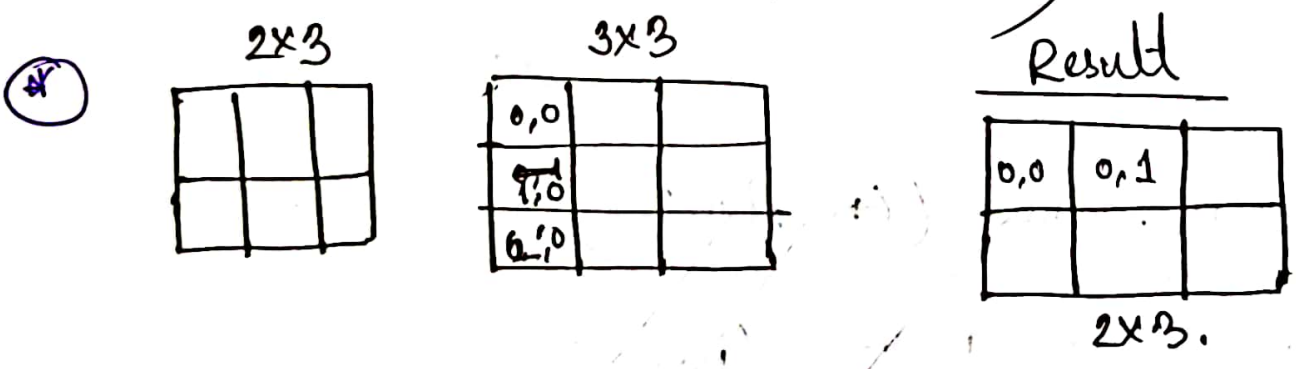
row major → last dimension stored first

9	-1	10	-2	11	-3
0x1	0x2	0x3	0x4	0x5	

multiplication

```
r1, c1 = m1.shape
r2, c2 = m2.shape.

if c1 != r2:
    print ("Cannot")
else:
    result = np.zeroes((r1, c2))
```



m1	m2	Result
0th row	0th col	(0,0)
0th row	1st col	(0,1)
mth row	nth col	(m,n)

Salicid™